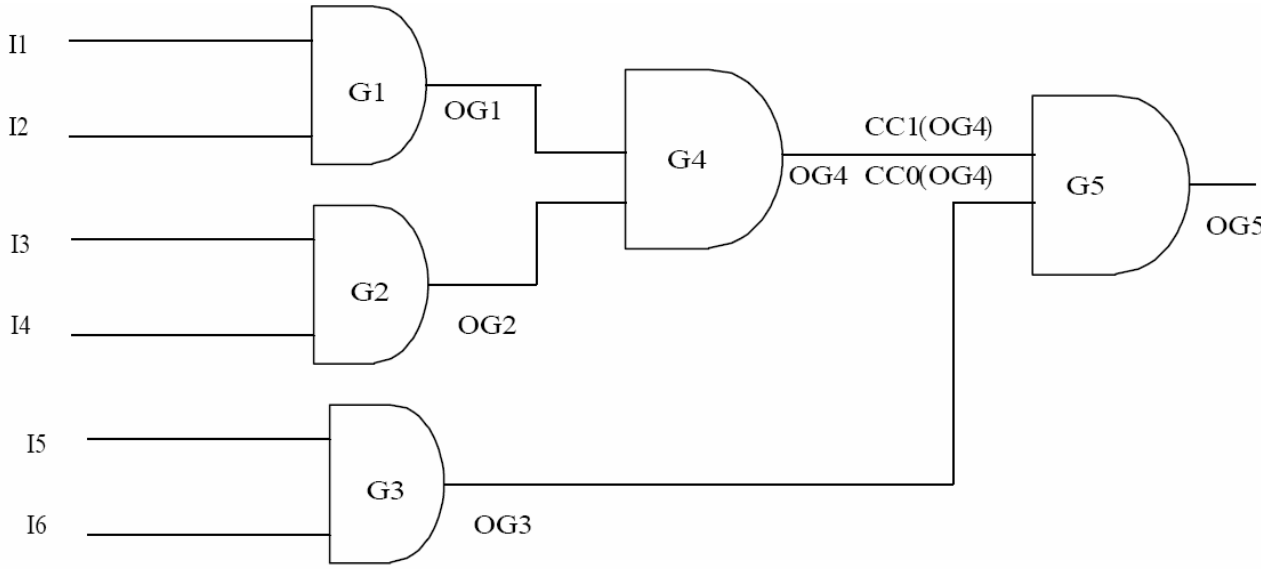
Module-VIII
Lecture-IV
Testability Measures (SCOAP)

1. **Introduction**

In the last lecture we learnt how to generate test patterns using fault simulation. Also we discussed that easy to test faults can be covered using fault simulation and for difficult ones, we need to use sensitize-propagate- justify approach. In this lecture we will discuss a quick heuristic based algorithm that can rank the faults by their difficulty in testing.

Let us understand "difficulty" in testing by an example. Let us consider the circuit given in Figure 1 and two faults (i) s-a-1 at OG4, (ii) s-a-0 at OG4 and study which fault among the two is easy to test.



CC1(OG4)----I1=1,I2=1,I3=1,I4=1

CC0(OG4)----Any combination except
I1=1,I2=1,I3=1,I4=1

Figure 1. Example: Difficulty in testing the faults

To test s-a-1 fault at OG4, we need to make signal value at OG4 to 0 (i.e., control OG4 to 0). Also, OG4 needs to be propagated to the primary output OG5 for observation; for this, OG3 should be 1. Similarly, to test s-a-0 fault at OG4, we need to make signal value at

OG4 to 1. Further, OG4 needs to be propagated to the primary output OG5 for observation; for this, OG3 should be 1. So, for both the faults to be observable at OG5 (primary output), OG3 needs to be made 1; so difficulty in observation is same for both the cases. Difficulty in testing comprises two parts namely, (i) difficulty in controlling and (ii) difficulty in observing. As in the present case, difficulty in observation is same for both the faults, we will compute the difficulty in controlling OG4 and then find overall difficulty in testing of the faults. From Figure 1, it may be noted that to test s-a-1 fault at OG4, we need to make signal value at OG4 to 0; this is represented by CC0(OG4) i.e., combinational controllability of 0 at OG4. CC0(OG4) is achieved for all combinations of inputs except I1=1,I2=1,I3=1,I4=1; so CC0(OG4) is achieved by 15 combinations of inputs I1,I2,I3,I4. On the other hand to test s-a-0 fault at OG4, we need to make signal value at OG4 to 1; this is represented by CC1(OG4) i.e., combinational controllability of 1 at OG4. CC0(OG4) is achieved for only one combination of inputs I1=1,I2=1,I3=1,I4=1. So it may be stated that to achieve the signal value to be controlled at OG4 to test s-a-0 fault is 15 times more difficult than to achieve the signal value to be controlled for s-a-1 fault. So, s-a-0 fault at OG4 is more difficult to test than s-a-1 fault at OG4.

It may be noted that in this procedure we determined which fault is more difficult to test, but at the same time also found a pattern to test it. For example, to test s-a-0 fault at OG4, test pattern is I1=1,I2=1,I3=1,I4=1,I5=1,I6=1;

- I1=1,I2=1,I3=1,I4=1 makes OG4 to 1 --*Sensitization*
- OG3 is 1—*Propagation*
- I5=1,IG=1—*Justification*

In other words, the scheme used in this example to rank faults on basis of their difficulty in testing is as complex as ATPG by Sensitization- Propagation-Justification. So we need a computationally simple (may be approximate) scheme to rate the faults by order of their difficulty in testing. In this lecture we will learn such an algorithm, which is approximate but computationally simple and can order all the faults (by their difficulty in testing) in two iterations of the circuit. This algorithm is called SCOAP--*Sandia Controllability/Observability Analysis Program.*

In SCOAP, each net *l* say, has three values associated, CC0(*l*),CC1(*l*),CO(*l*) representing controllability of 0 at *l,* controllability of 1 at *l* and observability of *l* at a primary output, respectively. Each net *l* has associated with it positive integers n_1,n_2,n_3 and represented as is (n_1,n_2)n_3, where CC0(l)= n_1,CC1(l) =n_2,CO(l)=n_3.

First, all the primary inputs are directly assigned CC0=1 and CC1 =1; it is assumed that as the primary inputs are directly controllable, to make their signal values 0 or 1 require "effort proportional to 1". Following that, CC0 and CC1 are determined level wise for the circuit using SCOAP rules for logic gates; if CC0 and CC1 values are known for all inputs of a gate then CC0 and CC1 for the gate output can be computed using some simple rules. In a similar way, combinational observability (CO) of all primary outputs is assumed 0; as primary outputs are directly observable, to see their signal values require an "effort proportional to 0". Following that, CO values are determined level wise for the circuit (now moving from primary outputs to primary inputs) using SCOAP rules.

Now, given a net *l* , difficulty to test a s-a-0 fault in it is proportional to CC1(*l*) (as we need to apply 1 in *l*) plus CO(*l*) (as we need to observe the value of *l* at a primary output). In a similar way, difficulty to test a s-a-1 fault at *l* is proportional to CC0(*l*) plus CO(*l*).

In the next two sections we will see the rules for computing CC0,CC1 and CO for logic gates.

2. **SCOAP Rules to Compute Combinational Controllability of 0 (CC0) and Combinational Controllability of 1 (CC1) for Logic Gates**

In this section we will study rules to determine CC0 and CC1 for logic gates.

Let us start with an AND gate with two inputs as **a**, **b** and output as **c**. To compute CCO(c)/CC1(c) we need to know CC0(a),CC1(a) and CC0(b),CC1(b). Now,

- To control the value of **c** to 0 (CCO(c)), either **a** is to be 0 or **b** is to be 0. So, CCO(c) is minimum of difficulty to control **a** to 0 or **b** to 0. Also we add 1 to CCO(c) as we progress by a level when we go from input to output of a gate. So, CCO(c) = min[CCO(a),CC0(b)] +1.
- To control the value of **c** to 1 (CC1(c)), both **a** and **b** are to be 1. So, CC1(c) is sum of difficulty to control **a** and **b** to 1. Also we add 1 to CC1(c) as we progress by a level when we go from input to output of a gate. So, CC1(c) = CC1(a)+CC1(b) +1.

CCO(c)/CC1(c) for NAND gate (two inputs as **a**, **b** and output as **c**) can be computed as follows

- To control the value of **c** to 0 (CC0(c)), both **a** and **b** are to be 1. So, CCO(c) is sum of difficulty to control **a** and **b** to 1, plus 1 for change in level. So, CC0(c) = CC1(a)+CC1(b) +1.
- To control the value of **c** to 1 (CC1(c)), either **a** is to be 0 or **b** is to be 0. So, CC1(c) = min[CCO(a),CC0(b)] +1.

CCO(c)/CC1(c) for OR gate (two inputs as **a**, **b** and output as **c**) can be computed as follows

- To control the value of **c** to 0 (CC0(c)), both **a** and **b** are to be 0. So, CC0(c) = CC0(a)+CC0(b) +1.
- To control the value of **c** to 1 (CC1(c)), either **a** is to be 1 or **b** is to be 1. So, CC1(c) = min[CC1(a),CC1(b)] +1.

CCO(c)/CC1(c) for NOR gate (two inputs as **a**, **b** and output as **c**) can be computed as follows

- To control the value of **c** to 0 (CCO(c)), either **a** is to be 1 or **b** is to be 1. So, CC0(c) = min[CC1(a),CC1(b)] +1.
- To control the value of **c** to 1 (CC1(c)), both **a** and **b** are to be 0. So, CC1(c) = CC0(a)+CC0(b) +1.

CCO(c)/CC1(c) for XOR gate (two inputs as **a**, **b** and output as **c**) can be computed as follows

- To control the value of **c** to 0 (CC0(c)), either both **a** and **b** are to be 0 or both **a** and **b** are to be 1. So, CC0(c) = min[(CC0(a)+CC0(b)), (CC1(a)+CC1(b))] +1.
- To control the value of **c** to 1 (CC1(c)), either **a** is 1 and **b** is 0 or **a** is 0 and **b** is 1. So, CC1(c) = min[(CC0(a)+CC1(b)), (CC1(a)+CC0(b))] +1.

CCO(c)/CC1(c) for NOT gate (input as **a** and output as **c**) can be computed as follows

- To control the value of **c** to 0 (CC0(c)), **a** is to be 1. So, CC0(c) = CC1(a)+1.
- To control the value of **c** to 1 (CC1(c)), **a** is to be 0. So, CC1(c) = CC0(a)+1.

CC0(c1), CC0(c2)…. CC0(cn)/ CC1(c1), CC1(c2)…. ,CC1(cn) for a fanout net with stem as **a** and **c1,c2,…,cn** as branches can be computed as follows

- As all the values of the branches can be controlled by the value at the stem CC0(c1)=CC0(c2)…. =CC0(cn)=**CC0(a)**. Similarly, CC1(c1)=CC1(c2)…. =CC1(cn)=**CC1(a).** As there is no change in level, 1 is not added.

These rules are summarized in Figure 2.

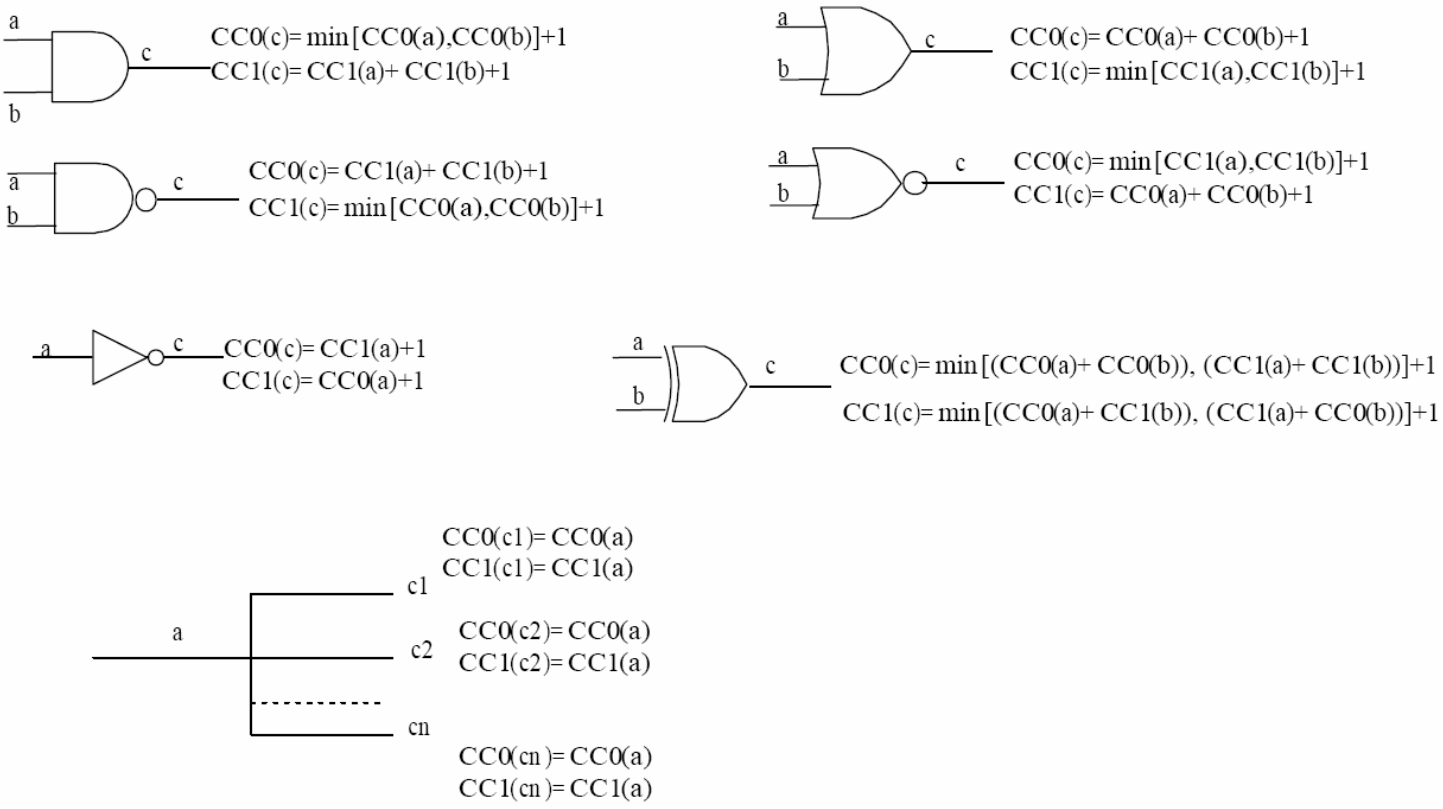


Figure 2. Summarization of controllability rules for logic gates.

3. **SCOAP Rules to Compute Combinational Observability (CO) for Logic Gates**

In this section we will study the rules to determine combinational observability (at primary output) for logic gates. It may be noted that for computation of observability for any net *l* say, we need not do it separately for *l=1* and *l=0*. This is because, for observability of *l* through a gate G say (where *l* is one of the inputs), inputs other than *l* of the gate must be non-controlling; this condition holds irrespective of *l=1* and *l=0*.

Let us start with an AND gate with two inputs as **a**, **b** and output as **c**. To compute CO(a) and CO(b) we need to know CO(c). Now,

- To observe the value of **a** at a primary output (CO(a)), **b** is to be 1. So, CO(a) is the difficulty to control **b** to 1 plus the observability of **c**. Also we add 1 to CO(a) as we progress by a level when we go from output to input of a gate. So, CO(a) = CC1(b) + CO(c)+1.
- To observe the value of **b** at a primary output (CO(b)), **a** is to be 1. Similar to computation of CO(a), CO(b) = CC1(a) + CO(c)+1.

CO(a), CO(b) for NAND gate (two inputs as **a**, **b** and output as **c**) can be computed as follows

- To observe the value of **a** at a primary output (CO(a)), **b** is to be 1. So, CO(a) = CC1(b) + CO(c)+1.
- To observe the value of **b** at a primary output (CO(b)), **a** is to be 1. Similar to computation of CO(a), CO(b) = CC1(a) + CO(c)+1.

CO(a), CO(b) for OR gate (two inputs as **a**, **b** and output as **c**) can be computed as follows

- To observe the value of **a** at a primary output (CO(a)), **b** is to be 0. So, CO(a) = CC0(b) + CO(c)+1.
- To observe the value of **b** at a primary output (CO(b)), **a** is to be 0. Similar to computation of CO(a), CO(b) = CC0(a) + CO(c)+1.

CO(a), CO(b) for NOR gate (two inputs as **a**, **b** and output as **c**) is same as that of OR gate and can be computed as follows

- CO(a) = CC0(b) + CO(c)+1.
- CO(b) = CC0(a) + CO(c)+1.

CO(a) and CO(b) for XOR gate (two inputs as **a**, **b** and output as **c**) can be computed as follows

- To observe the value of **a** at a primary output (CO(a)), **b** is to be 0 or 1; if **b** is 1 then **c**=**a** and if **b** is 0 then **c**=NOT(**a)**. So, CO(a) is the minimum of (difficulty to control **b** to 1 or difficulty to control **b** to 0) plus the observability of **c**. Also we add 1 to CO(a) as we progress by a level when we go from output to input of a gate. So, CO(a) = min(CC0(b),CC1(b)) + CO(c)+1.
- Similar to computation of CO(a), CO(b) = min(CC0(a),CC1(a)) + CO(c)+1.

CO(a) for NOT gate (input as **a** and output as **c**) can be computed as follows

- There is only one input in a NOT gate and it is always observable at the output (i.e., **c**=NOT(**a**)); so C0(a) = CO(c)+1.

CO(a) for a fanout net with stem as **a** and **c1,c2,…,cn** as branches can be computed as follows

- As all the values of the branches can be observed at the stem, C0(a) = min[CO(c1), CO(c2)…,CO(cn)]. As there is no change in level, 1 is not added.

These rules are summarized in Figure 3.

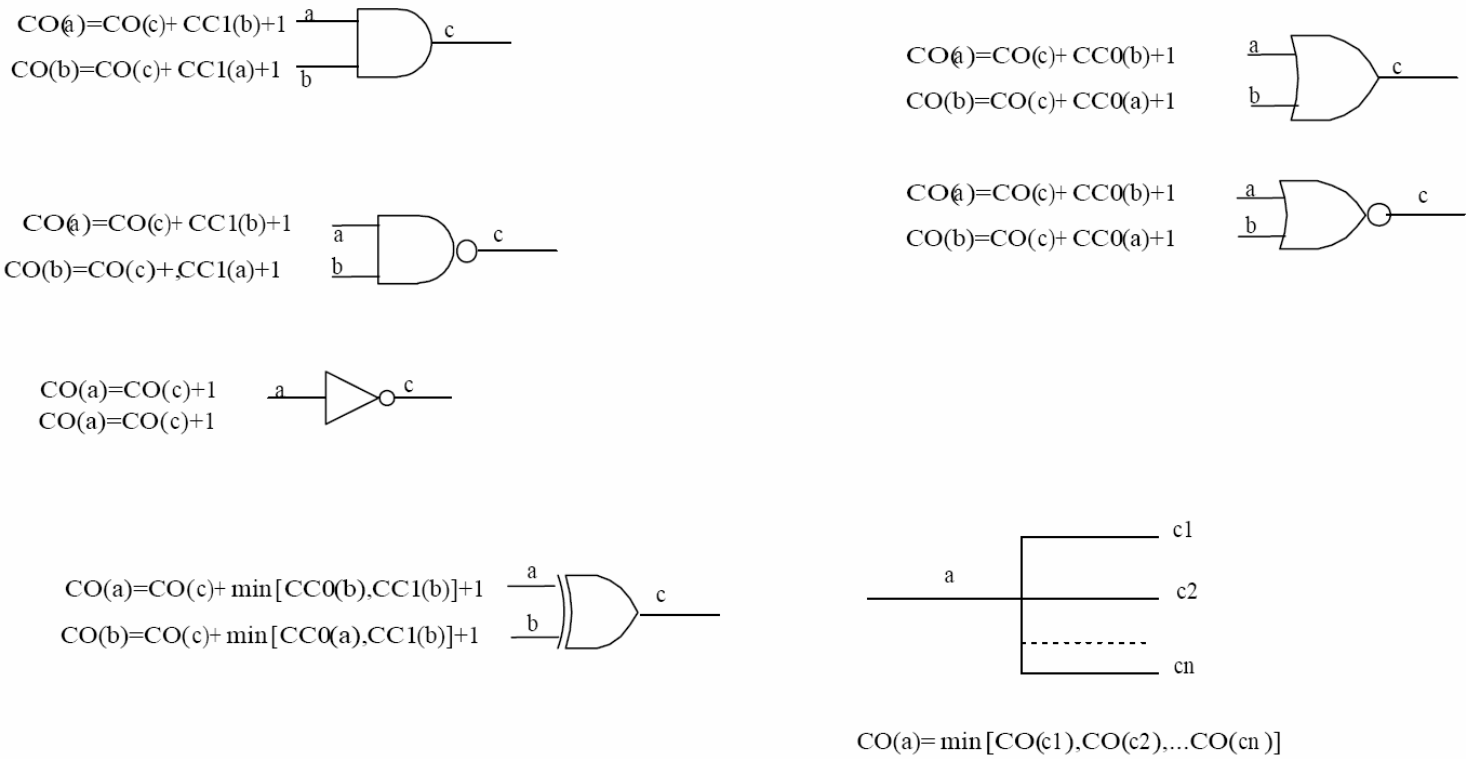


Figure 3. Summarization of observability rules for logic gates.

In the next section we will study how to determine CC0, CC1 for a simple circuit. Once all the controllability values are determined we will go for finding the observabilities of the nets of the same circuit.

4. **Combinational Controllability Calculation for a Simple Circuit**

Consider the simple circuit in Figure 4. We will show step wise calculation of controllability values at each net in the circuit.

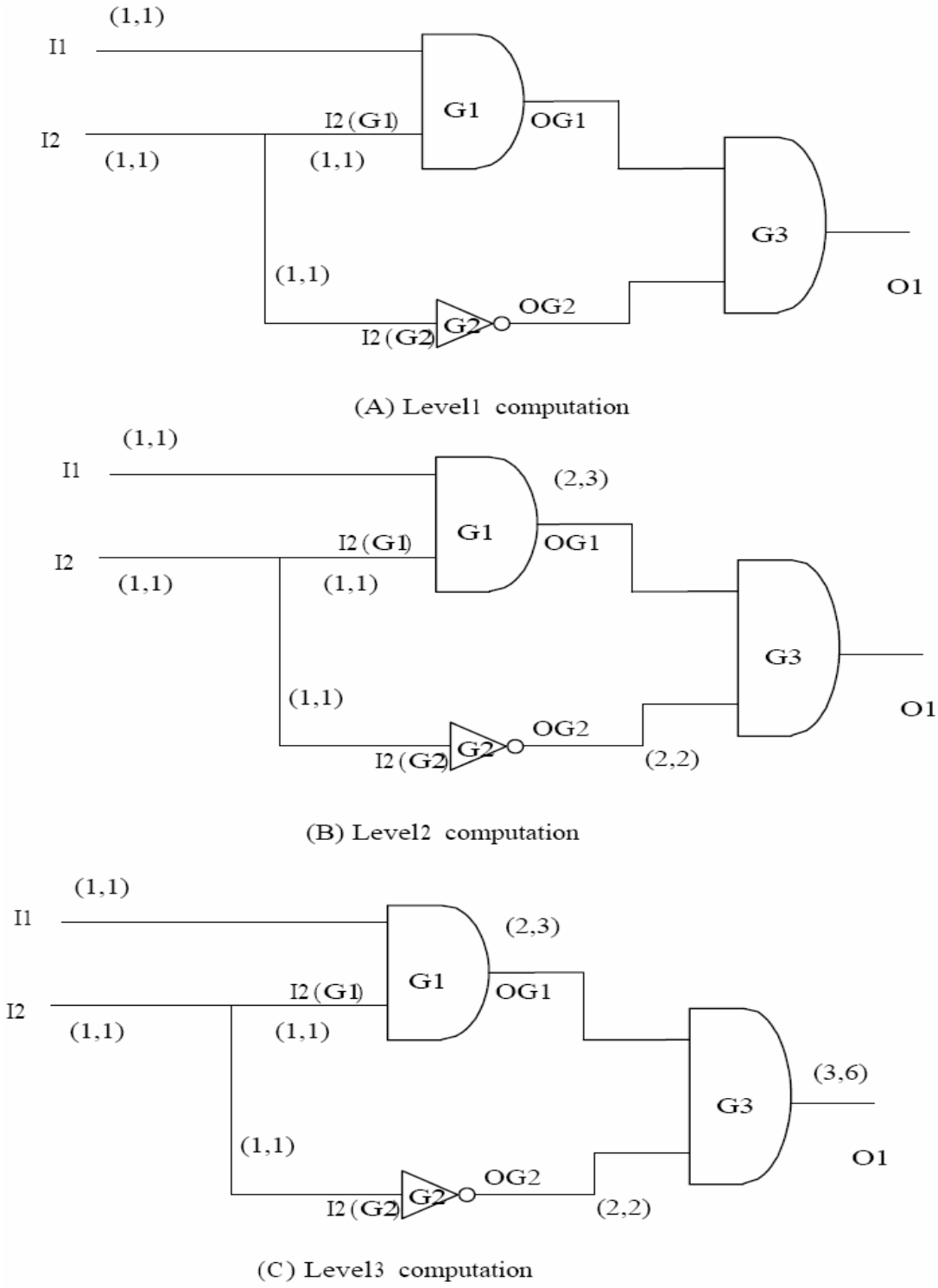


Figure 4. Step wise calculation of controllability values

- Level 1: All the primary input nets (I1,I2) and their fanout branches (I2(G1) and I2(G2)) are directly assigned CCO=1 and CC1=1; this is illustrated in Figure 2(A) as (1,1) in the respective nets.
- Level 2:
    - CCO and CC1 for OG1

      CC0(OG1) = min[CCO(I1),CC0(I2(G1))] +1

      =min[1,1]+1=2

      CC1(OG1) = CC1(I1) + CC1(I2(G1)) +1

      =1+1+1=3

      CCO and CC1 for OG1 is illustrated in Figure 2(B) as (2,3) marked in OG1 .

    - CCO and CC1 for OG2

      CC0(OG2) = CC1(I2(G2))]+1

      =1+1=2

      CC1(OG2) = CC0(I2(G2)) +1

      =1+1=2

      CCO and CC1 for OG2 is illustrated in Figure 2(B) as (2,2) marked in OG2 .
- Level 3:
    - CCO and CC1 for O1

      CC0(O1) = min[CCO(OG1),CC0(OG2))] +1

      =min[2,2]+1=3

      CC1(OG1) = CCO(OG1)+CC0(OG2))+1

      =3+2+1=6

CCO and CC1 for O1 is illustrated in Figure 2(C) as (3,6) marked in O1

5. **Combinational Observability Calculation for a Simple Circuit**

Figure 5 shows step wise calculation of observability values at each net in the circuit of Figure 4.

- Level 1: The primary outputs (O1) is directly assigned an observability value of 0; this is illustrated in Figure 5 (A) by a 0 (after (3,6)) at O1.
- Level 2:
  - CO for OG1

    C0(OG1) =CC1(OG2) +CO(O1)+1

    =2+0+1=3
  - CO for OG2

    C0(OG2) =CC1(OG1) +CO(O1)+1

    =3+0+1=4

  CO for OG1 and OG2 are marked as 3 and 4 in Figure 5 (B) in the respective nets after their CC0,CC1 values.
- Level 3:
  - CO for I1

    C0(I1) =CC1(I2(G1)) +CO(OG1)+1

    =1+3+1=5
  - CO for I2(G1)

    C0(I2(G1)) =CC1(I1) +CO(OG1)+1

    =1+3+1=5
  - CO for I2(G2)

    C0(I2(G2)) = CO(OG2)+1

    =4+1=5

  CO for I1, I2(G1) and I2(G2) are marked as 5, 5 and 5 in Figure 5 (C) in the respective nets after their CC0,CC1 values.
- Level 4:

  CO for I2 = min[CO(I2(G2)), CO(I2(G1))]

  =min[5,5]=5

  CO for I2 is marked as 5 in Figure 5 (D) in the respective net.

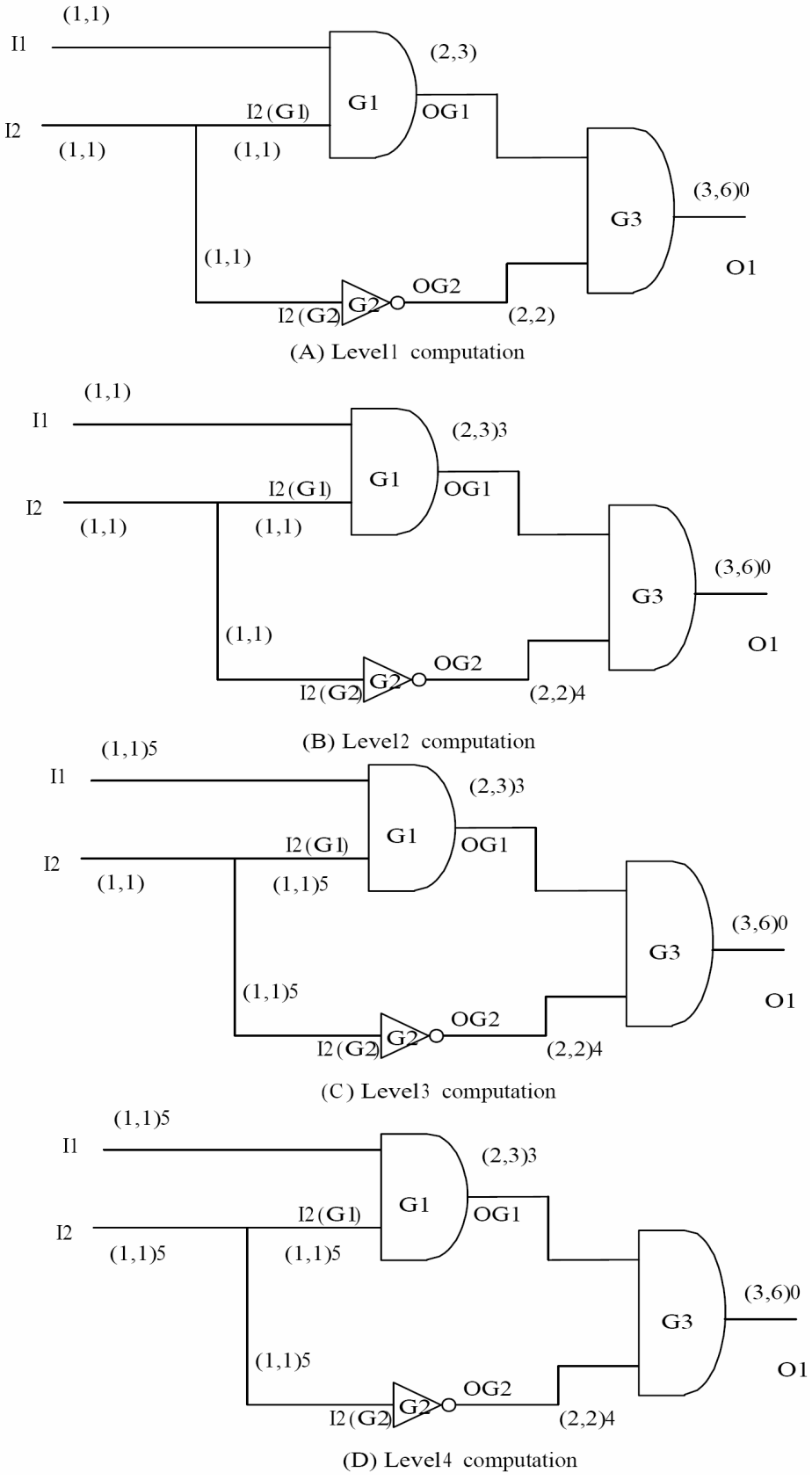


Figure 5. Step wise calculation of observability values

By the example we saw how to mark controllability and observability values in all the nets of a circuit in two level wise iterations. Once these values are calculated at all the nets, difficulty to test s-a-0 (s-a-1) faults in these nets can be determined as a sum of their CC1 (CC0) and CO values. Following that a threshold can be determined and all faults whose difficulty values are above the threshold can be targeted for "sensitization–propagation-justification" based ATPG. Remaining faults are easier to test and test patterns can be generated for such faults using fault simulation.

Question and Answers:

1. SCOAP is a fast heuristic to compute difficulty in controlling and observing signals in a net in a circuit. Why is the method approximate/not accurate?

   Answer: In SCOAP all the branches of a fanout net are considered independent. In the circuit given below, primary output (G2's output) can never have the value of 1 because of dependency of inputs of G2, which in turn arises because of the fanout at the input. However, CC1(G2) is 6 and not infinity.

   Similar type of inadequacies arise for observabilities.

